LAB 3: IDENTIFY CODE SMELL AND REFACTOR

Aims:

- Identify code smell and refactor a small piece of software.

Instruction

For this lab, you will start with a working program to play the game noughts and crosses against a computer. Your task will be to observe the code smells described in each section below, and gradually implement the identified refactoring. We expect that this lab will take you at least 2 weeks to 3 weeks to complete.

If you log into https://projects.cs.nott.ac.uk/ you should see a group called 'COMP2013 Labs', inside is a subgroup named after your username, and inside that is a project called COMP2013Labsheet. You should clone this repository and work on it (DO NOT FORK to your own namespace). You might want to use resources in this site : https://www.jetbrains.com/help/idea/gitlab.html to work on your project in IntelliJ.

When complete, you should submit the "Project ID" to moodle.

Useful Terms

- Attribute and field are both terms for variables that belong to objects (rather than local to a method). Attribute is the generic term used in object-oriented programming, while field is the term that Java uses. In this case, they mean the same thing.
- The `static` keyword means that the method or field it's attached to belongs to the class, not to a specific object.

Part 1. Object-Oriented Code

Overview

The first code smell to note is the use of the keyword **static**. Any method or variable defined as static belongs to the class, not to objects. When code doesn't use objects, we refer to it as **procedural programming**, and we want to use **object-oriented programming**, because procedural code is less modular, more prone to errors, and more difficult to fix.

Goal

When you are done with this part of the lab, you will only have one use of the keyword `static`, which will be your main method. And your `main()` method will be short.

Tasks

1. Start by removing the `board` variable from the main method
   a. Delete `board` from the `main()` method
   b. Add it as a private (non-static) field to the `NoughtsAndCrosses` class
      Your `NoughtsAndCrosses` class should now have this line of code in the class (not in a method).
      ```
      private char [][] board;
      ```

2. When you do this, the code will no longer compile. The `main()` method cannot see the `board` variable, because the `main()` method is `static` and the field is not. We want to use static as little as possible, so follow these steps to fix it.
   a. Since this method will be the major game loop, rename the `main()` method to `play()`
   b. Remove the `static` keyword
   c. Remove the `String [] args` parameter from the method signature
      (but leave the parentheses)

   With just these two changes, the code should now compile again. But since we no longer have a `main()` method, the code can't be run.

3. We need to add a `main()` method to run the code.
   a. Go to the bottom of the class
   b. Add an empty main method, which you will use to create objects and start the game:
      ```
      public static void main(String [] args) {

      }
      ```
   c. Verify that you can compile and run the code
      … but it won't do anything.

4. Now we need to add code to the `main()` method we just created
   a. Since we are using a class, create a new `NoughtsAndCrosses` object
   b. On the next line, call `play()` on the object
   c. Run the code
      When you run the code now, you'll get an error. What is the error? What does it mean?

5. Since we are working with a true class now, we should create a constructor, a special method that will be used to create the object and set up any fields it needs.
   a. Create a constructor
   b. In the constructor, create the array and store it in the field:
      ```
      board = new char[3][3];
      ```

6. The game should be playable. Run the game and verify that it works.

   We still have three uses of `static` in the code, and we want that at the minimum (just one use, for the `main()` method). The `static` keyword means that the method or field belongs to the class, not an object, so if we use `static`, we are breaking the object orientation of the code. Sometimes `static` is useful or even necessary, but we want to avoid it if possible.

   Also, we want to use the `board` field, rather than pass it around as a parameter.

7. We want to use the `board` field, not a parameter
   a. Remove the `board` parameter from the `printBoard()` method
   b. Remove the `board` parameter where `printBoard()` is called in the `play()` method

8. Now we have a problem. The Java compiler is telling us that the `board` field cannot be accessed from a static context. Translated, that means that the `printBoard()` method is still `static`, so it belongs to the class. It cannot see the `board` field, because each separate `NoughtsAndCrosses` object has its own `board`.
   a. Remove the keyword `static` from the method signature

9. We need to change one more thing in `printBoard()`.
   In this program, we are representing the noughts and crosses board with a two-dimensional (2D) array of the char datatype, with three rows of three columns each.

   Recall that accessing individual elements of a 2D array in Java requires using the index operator `[]` twice, first for the row, then for the column. As an example, since our array is named board, we can access the top right element as below, setting it to hold the character 'X':
   ```
   board[0][2] = 'X';
   ```

   Because we are just reading the contents of the array, we can replace the `for` loop with an enhanced `for` loop. This works because a 2D array in Java is really just an array of 1D arrays. So we're peeling off each row as a separate array. If we do this, then this code:
   ```
   for(int row = 0; row < row.length; row++) {...
   ```
   becomes
   ```
   for(char [] chars: board) {...
   ```

   We don't even need to type any code!
   a. Click on the word `for` (to move the cursor there)
   b. You will notice a yellow lightbulb appear to the right of the line of code
   c. Click on that lightbulb
   d. Select the first option, which should be *Replace with enhanced 'for'*

10. But `chars` is a terrible variable name. It tells us the data type (which we already know), but not what the variable contains. Since each of these 1D arrays is one row of the board, let's change the variable name from `chars` to `row`.

But if we change it by typing, then we will have to change it multiple places. The most commonly-used refactoring is Change Name, and that has been automated.

a. Click on the name `chars` anywhere it is used (not on `char`!)
b. Go to Refactor->Change name
c. Type in the new name (`row`) and press Enter/Return

11. What is the benefit of all of this work? Object-oriented code is more self-contained and more modular. That means it can be more easily used and re-used. Before doing this work, we would need to run the code again to play a second game. Now we can just reuse our class.
    a. In the `main()` method, create a second `NoughtsAndCrosses` object
    b. Call the `play()` method on the new object
    c. Run the program and verify that you can now play two games, one after the other

PART 2. REFACTORING

Overview

Now that we have object-oriented code, we are not done. We still want to improve code quality by identifying **Code Smells** and using **Refactoring** to improve the code.

The most basic code smells are **Long Class** and **Long Method**. We have both. We have only one class, and we have one long method, `play()`, which is more than 60 lines long.

A **Long Method** usually tries to do too much, and is difficult to debug and often impossible to write **unit tests** for. A **Long Class** often violates the **Single Responsibility principle**. In this case, the `NoughtsAndCrosses` class represents the game rules, the players, and the board.

2a. Introduce Method

Overview

Identify some code within our **Long Method** that solves a particular problem, and place it in a separate method, then calling that method. This will simplify the code and make our methods shorter.

Tasks

Notice that early in the `play()` method, we have nine lines of code to set up the board. Since we know what this code does, we can use the **Introduce Method refactoring** to isolate it.

1. Highlight the lines of code that assign values into the `board` field
2. Select Refactor->Extract/Introduce->Method from the menu
3. Type in a name for the method; since the method resets the board, let's call it `resetBoard()`
4. Press the Enter/Return key
   IntelliJ will create the method and replace the code with a call to our new method

2b. Eliminate Repeated Code

Overview

Eliminate **Repeated Code** to reduce the likelihood of bugs, make the code easier to test, and make the code more flexible for future operations.

Information: Using the Java `char` Datatype

Remember that the `char` datatype in Java is an integer datatype. Just like the Java datatype `short`, it stores sixteen bits. The only time when it works differently than short is when we print it; instead of printing the numeric value, Java will print the corresponding character from ASCII or Unicode.[1]

As an example, this code would **NOT** print out a 0:

```
char x = 0;
System.out.println(x);
```

It will print out a null character, which won't show on the screen.

If we want to print a zero, we can do this:

```
char y = '0';
System.out.println(y);
```

Or this:

```
char z = 48; // 48 is the ASCII value for the '0' character
System.out.println(y);
```

Tasks

The updated `resetBoard()` method has one central problem: **Duplicate Code.** In this case, the code can be replaced by nested loops, because we are working with 2D data. We cannot use enhanced `for` loops because we're trying to change the data in the array.

1. Delete everything in the `resetBoard` method except for the first line
2. Comment out that line (but we will keep it for reference)
3. Write the outer `for` loop; name your loop variable `row` (rather than `i`)
4. Inside that, write the inner `for` loop; name your loop variable `col` (rather than `j`)
5. Assign a value into the array; for now, it's fine to always use '0'
6. Replace the array indexes (in the line you just wrote) with the correct loop variable
7. Even though we aren't done, let's test the code
   If we've done this correctly, the board should print out with a 0 in each cell of the board. You can stop the game after you test that.

Now we have another problem. We have a **Magic Constant** (or **Magic Literal**), which sets every cell to hold 0. We're going to **Replace Magic Literal** by creating a variable.

8. Now we're setting every single cell to the character '0'. That value has to change. We use variables to change values, so we will introduce a new variable.
   a. Click on the '0' and go to Refactor->Replace/Introduce-> Variable
   b. Name that variable `c`

[1] And that char won't store negative values.

The problem is that even now, we will still assign a '0' character to each cell. We need to change that, which is easier than it might seem.

9. Add the increment operator inside the loop
   `c++;`
10. Run your code to test it

This didn't work! Everything should still print out as the `0` character. The variable is getting created and destroyed repeatedly, because its scope is wrong (it's being created in the wrong place).

11. Move the `c` variable above the outer loop
12. Test and make sure that the code works

2c. Introduce Method and Remove Repeated Code Pt. 2

Overview

We still have a **Long Method** that is trying to do too much. In this case, a block of code checks for a winner. That should be isolated into a separate method. This will make the `play()` method easier to read and make the new method testable. Again, we'll use the **Introduce Method** refactoring.

Tasks

1. Highlight the lines of code that determine if the current player has won
2. Go to Refactor->Extract/Introduce->Method
3. IntelliJ may suggest `isGameOver()`, but that's *not* a good name; this is specifically checking for a win, and games can also end in ties. Call the method `checkWin()`
4. As with `resetBoard()`, the lines of code repeat and can be replaced by loops
   a. Create a loop to compare the values at column indexes 0, 1, and 2, changing only the row If all three are the same, return true
   b. Create a loop to compare the values at row indexes 0, 1, and 2, changing only the column If all three are the same, return true
   c. Note that these can be the same loop; if you need to write both and then see the similarity, that's fine, but you should end with a single loop and an if/else
5. Don't replace the the diagonal checks with loops, but they should return true if all three are the same

Now we have a new problem: in the method signature, the parameter `gameOver` is gray. That means it's an **Unused Variable**, which is another code smell, related to **Dead Code**. If we just delete it here, we will have an error in the `play()` method when we call this method, so let's use the IDE to clean up for us.

1. Click on the parameter name
2. Go to Refactor->Change Signature
3. In the list of parameters, make sure that `gameOver` is highlighted
4. Press the minus (−) button above to remove it
5. Press Refactor

2d. Introduce Class

Overview

We still have one really long class, `NoughtsAndCrosses`. Look at the list of methods. Do you notice anything about them:

- `play()`
- `resetBoard()`
- `isValidMove()`
- `printBoard()`
- `checkWin()`

Four of these methods are specific to the board. That sounds like a separate class, so we will use the **Introduce Class** refactoring.

Why do we want a separate class? That comes from the S in SOLID. The first rule is that classes should follow the **Single Responsibility Principle**. Right now, the `NoughtsAndCrosses` class manages the gameplay and rules, the players, and the board, which are all separate responsibilities.

Tasks

Let's create a new class called `Board` to fix that.

1. Go to File->New->Java Class
2. Call the new class `Board`
3. Move (cut and paste, don't copy) the `board` field to the `Board` class
4. Move (cut and paste, don't copy) the methods `resetBoard()`, `isValidMove()`, `printBoard()`, and `checkWin()` to the `Board` class

At this point, `Board` should compile without problems, but we need to modify `NoughtsAndCrosses` to use the new `Board` class.

1. Add a new field of type `Board`, and call it `board`.
2. In the constructor, change the `board` field to point to a new instance of the `Board` class
3. Every time that the `play()` method calls a method that's now in the `Board` class, we need to call that method on the `board` field, so make that change
    a. You should make five changes, adding `board`. in front of calls to the methods we've moved

    If you've followed instructions properly, you should have a new problem. Some of the methods in `Board` are `private`, and private methods are only visible to the class itself.
4. Change all of the methods in `Board` to be `public`

Now we have just one more problem preventing us from compiling and running, which is that we directly access the board to set the move.

Since this is a useful operation, let's extract a method of the three lines

- The line that calculates the row
- The line that calculates the column
- The line that actually sets the value in the 2D array

1. Highlight all three lines
2. Go to Refactor->Extract/Introduce->Method

3. Name the method `setMove()`

Note that this doesn't solve the problem, but it does isolate it. Think about this method: does it belong in the `NoughtsAndCrosses` class or the `Board` class? Let's move it to the `Board` class (just cut and paste).

1. Highlight the method
2. Go to Refactor->Move Instance Method
3. Select to move the method to the `Board` class
4. The refactoring tool makes an incorrect assumption, so in the `Board` class, change the code to assign to the `board` field, rather than `this`

At this point, your code should both compile and run.

PART 3. ISOLATING PLAYERS

Overview

As noted, the `NoughtsAndCrosses` class still has multiple responsibilities, and we want each class to have a **Single Responsibility**. In addition, within the play() method, we see the **Duplicate Code** code smell; there are only minor differences in the code for the players changing their moves. We will keep using the **Introduce Method** and **Introduce Class** refactorings.

Separately, our computer player algorithm just randomly guesses a move. We will address that in a later lab.

Part 3a. Introduce `getMove()` Method

Overview

Because this is object-oriented programming, we want to use fields when sharing data, instead of parameters. When we pass around data unnecessarily, we call that the **Tramp Data** code smell. If we move the computer player code into its own method, we would need to pass the `Random` object as a parameter. Instead, we can make it a field.

Tasks

1. At the top of the `play()` method, highlight the code for the `Random` variable
2. Go to Refactor->Extract/Introduce Field
   NOTE: don't press Enter/Return yet!
3. Select "Initialize in: field declaration"
4. Press Enter/Return

Second, we're going to **Introduce Method** isolate the code for a computer move.

1. Highlight the `while(true)` loop for the `computer` in the `play()` method
2. Go to Refactor->Extract/Introduce->Method
3. Name the method `getMove()`

Part 3b. A `ComputerPlayer` Class

Overview

All of the changes above can be performed manually with identical results. Using the automated refactoring tools helps ensure that we always have compilable code. For this part, we're going to do some manual refactoring, because it's not automatically supported.

Tasks

1. Create a new class called `ComputerPlayer`
2. Cut and paste the `Random` field from `NoughtsAndCrosses` to `ComputerPlayer`
3. Move the `getMove()` method from `NoughtsAndCrosses` to `ComputerPlayer`

We now have a problem; since the `board` field is in the `NoughtsAndCrosses` class, and we don't have an instance of `ComputerPlayer`.

4. In `ComputerPlayer`, add a `Board` parameter (`Board board`) to the `getMove()` method
5. In the `NoughtsAndCrosses.play()` method, add the parameter (`board`) to the call
6. Add a new field to `NoughtsAndCrosses` for the computer (`ComputerPlayer computerPlayer`)
7. Initialize that field in the constructor
8. Prefix the call to `getMove()` by adding `computerPlayer.` in front
9. The compiler will warn you that the method `getMove()` is marked `private`, so switch it to `public`
10. Test

Part 3c. A `HumanPlayer` Class

Overview

Now that we have a working class and method for ComputerPlayer, we need to repeat the steps above to create a class for HumanPlayer. Follow the steps above, but note that there will be some differences; for example, you'll have to move the Scanner field to the new class.

Tasks

1. Make the `Scanner` variable a field
2. Extract a `getMove()` method from the code for a human player
3. Create a `HumanPlayer` class and move the `Scanner` field and the `getMove()` method there
4. Add a `Board` parameter to the `getMove()` method in HumanPlayer
5. In the `NoughtsAndCrosses` class, `HumanPlayer` field and initialize it
   `private final HumanPlayer humanPlayer = new HumanPlayer();`
6. Add a `Board` parameter to the `getMove()` method in `HumanPlayer`
7. In the `NoughtsAndCrosses.play()` method, add the `humanPlayer.` prefix to `getMove()`
8. Add the `board` parameter
9. Make `HumanPlayer.getMove()` a `public` method
10. Test

Part 3d. Cleanup

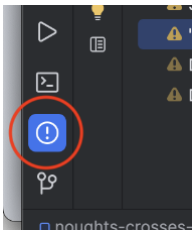
Overview

Let's do some cleanup to get rid of repeated code and warnings, and better isolate some functionality. Periodically, it's a good idea to review any warnings and address them when possible. We will start with the `Board` class.

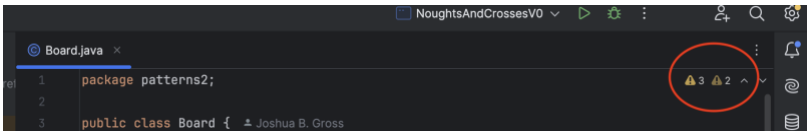
We want to eliminate warnings because they are suggestions of possible problems or possible improvements. For each warnings, we have a choice:

1. Use one of the suggested changes
2. Rewrite the code manually to address the warning
3. Add an annotation to the code to ignore the specific warning

- To see all warnings, you can either use the Problems button near the bottom left of the build tool



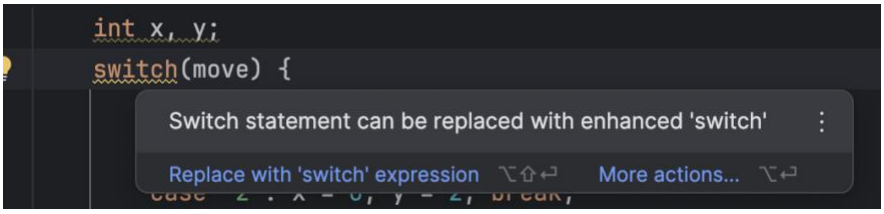
- OR you can click on the warning icon at the top right of the file.



Tasks

1. The first warning is that we can change the board field to be final. This might seem odd, since we will be changing the board as the game progresses, but it's correct. Think about what the keyword final does.
   a. Accept the suggested change.

2. The next warning is suggesting that we use an enhanced switch statement; hover over the warning to see the pop-up dialog box with that suggestion



   a. Click on "Replace with 'switch' expression and look at the code
      It's not really any clearer, is it? Use Undo (Cmd-z on Mac, Ctrl-z on Windows)
   b. Instead, let's fix this code. We know that there's a pattern here, so let's replace this code with an algorithm. We know that the ASCII/Unicode value for the char will be 48 higher than the number, so a value of 48 is 0, 49 is 1, 50 is 2, etc. Before writing the code, do some test arithmetic on paper to make sure that you implement this correctly.

        i. Create a new int variable to hold the move number, but subtract 48 from it
        ii. Use the division operator to determine the row (what should you divide by?)
        iii. Use the remainder operator to determine the column
    c. Test

3. The next warning is to simplify the conditional
    a. Accept this suggestion and review the change
    b. What changed? Do you prefer the code this way?
    c. If you don't like the code this way, you can undo the change
    d. To tell IDEA to ignore the warning for this section of code
        i. Click somewhere on the line
        ii. Click the yellow lightbulb
        iii. On the popup window, next to "Simplify", press the three dots
        iv. Click "Suppress for statement", and IDEA will insert an annotation

4. Check `ComputerPlayer`, which has one warning, the use of break inside of a loop
    a. Click on the line
    b. Click on the yellow light bulb
    c. Select "Move condition to loop", and note that the loop is changed to a do-while loop
    d. Repeat this for `HumanPlayer`

5. Repeat for `NoughtsAndCrosses`
    a. Make all of the fields `final`
    b. The remaining warning is that the args parameter to main is not used. We could remove this parameter, but then the code would not work with older versions of Java
    c. In the Warnings window at the bottom, right-click on "Parameter 'args' is never used"
    d. In the dialog box that comes up, click on "Show quick fixes"
    e. In the dialog box that comes up, click the three dots
    f. Select "Suppress for class"
    g. Note that the annotation gets added to the very top of the class

PART 4. An Inheritance Hierarchy

Overview

The `HumanPlayer` and `ComputerPlayer` classes are very different internally; they have different algorithms and fields. But from an external perspective, they're identical. We're going to use inheritance to connect the two, implementing the **Extract Superclass** refactoring.

Part 4a. Create the Superclass

Overview

First, we will create a superclass that will connect the two player classes.

Tasks

1. Create a class called `Player`

2. In the class, add the keyword `abstract` after between `public` and `class`
3. Add an `abstract` method for `move()`
   `public abstract void move(board b);`
4. Note that this method has two differences from our current method:
   a. Its return type is `void` (rather than `char`)
   b. It is named `move()`, not `getMove()`

   It doesn't make sense to pass a move back to the play method, since the player can directly control the board
5. In `HumanPlayer`, add the `extends` relationships to `Player`
6. Change the name of the method to `move()`
7. Change the return type of the method to `void`
8. Remove the line returning the `move` variable
9. Add a line at the end of the `move()` method to change the board by calling `board.setMove()`

Part 4b. Add a `moveCharacter` Field

Overview

Notice a problem. In order to actually make the move, we need to know whether this player is the noughts (the letter O) or the crosses (the letter X). We have two choices here. We can always have this player be X or O, or we can set this earlier.

What we don't want to do is leave that information only in `play()`. In general, the `NoughtsAndCrosses.play()` method should ensure that the game loops correctly and that it ends, but nothing more. That's what we have been working on: making sure that each class controls a **Single Responsibility**.

Tasks

1. In `Player`, add a new field
   `private final char playerCharacter;`
2. Add a constructor that takes one parameter, `playerCharacter`, and assigns it to the field (remember, you must use the prefix `this.` to note the field, rather than the parameter)
3. Add a method called `getPlayerCharacter()` that returns the `playerCharacter` value
4. In the constructor for `HumanPlayer`, add a parameter for `playerCharacter`
5. Call the constructor on the superclass to pass the parameter
   `super(playerCharacter);`
   Remember that because it is marked private, subclasses of `Player` cannot directly access the `playerCharacter` field. Also remember that this must be the first line of the constructor
6. In the call to `board.setMove()` in the `move()` method, add a call to `getPlayerCharacter()` as a parameter

At this point, there should be no compiler errors in `HumanPlayer`, but there will be in `NoughtsAndCrosses`, so we can't compile and test yet.

Before we fix those errors

7. Repeat the steps above for `ComputerPlayer`, making it a subclass of Player
8. Make sure that `Player`, `ComputerPlayer`, and `HumanPlayer` have no errors before proceeding

Part 4c. Fixing `NoughtsAndCrosses`

Overview

Finally, we want to remove the unnecessary logic from the game loop (the `play()` method) in `NoughtsAndCrosses`.

Tasks

1. Where we instantiate the ComputerPlayer and HumanPlayer objects In the constructor, add the appropriate character (X or O) as a parameter
2. Inside the game loop, where the code currently gets the move from the correct player and then sets it, change the code to just call the `move()` method, so this code:

   ```
   if (currentPlayer == user) {
       move = humanPlayer.getMove(board);
   } else {
       move = computerPlayer.getMove(board);
   }
   board.setMove(move, currentPlayer);
   ```

   becomes

   ```
   if (currentPlayer == user) {
       humanPlayer.move(board);
   } else {
       computerPlayer.move(board);
   }
   ```

3. Get rid of the `move` variable
4. Test your code

Part 4d. Simplifying `NoughtsAndCrosses` One More Time

Overview

This code is greatly simplified. When we started, the game loop took 30 lines of code to do what it now does in five. A nice improvement… but we can make it even simpler. This will follow the **Use Polymorphism** refactoring, which is why we introduced inheritance.

Tasks

1. At the top of the `play()` method, remove the user and computer variables
2. Change the datatype of the `currentPlayer` variable to `Player`
3. Assign it the value of `humanPlayer`

4. Now we don't need the conditional around the calls to move() at all, so this code:

```
if (currentPlayer == user) {
    humanPlayer.move(board);
} else {
    computerPlayer.move(board);
}
```

becomes

```
currentPlayer.move(board);
```

5. We need to fix one more thing; right now, our code to switch players is dependent on char
6. When comparing two objects, we don't use ==, we use `.equals()`, so our condition should be `if(currentPlayer.equals(humanPlayer)) {...`
7. Assign the correct `Player` subclass (`humanPlayer` or `computerPlayer`)
8. Test

Part 4e. Cleaning Up Player

Overview

You may have noticed when playing the game that this:

```
X's turn
```

Has become something like

```
HumanPlayer@38af3868's turn
```

We can fix that!

Tasks

1. In the `Player` class, to to *Code->Override Methods*
2. Select `toString():String`
3. Replace this automatically created line
   `return super.toString();`
   with
   `return getPlayerCharacter();`
4. But that doesn't compile! Why? We're returning a `char`, but Java wants a `String`
5. One way to fix this is to coerce the `char` into a `String` by changing the line to
   `return getPlayerCharacter + "";`
6. Test your code thoroughly

5. Wrapping Up

Finally, since you are going to be expected to use **Code Smells** and **Refactoring** in your own work:

1. Review this document, looking for bold text noting **Code Smells** and **Refactoring**
2. For each **Code Smell**, identify the specific code that caused the concern
3. For each **Refactoring**, identify the change and why it was valuable

Make sure this information is listed in your project README.